Replay Court Specification v0.3

> Implementation-grade. Supersedes v0.1 and v0.2 entirely. All locks applied. This document is the normative reference.

---

0. Constitutional Rule

The Replay Court must be built before autonomous execution.

Every later system depends on this invariant:

> No state transition is real unless the Replay Court can independently verify it.

That is the bedrock.

---

1. Cryptographic Primitives (Locked)

Primitive	Choice	Rationale
Hash function	`keccak256`	EVM-native, Solidity-compatible
Signature scheme	`secp256k1` recoverable signatures	Aligns with `ecrecover`, Foundry, cross-chain settlement
ABI encoding	Strict Ethereum ABI	Deterministic, no padding variation

All signatures in this system are secp256k1 recoverable signatures over keccak256 digests.

Specific rules:

- Governor signatures: secp256k1 over `transition_id`
- Proposer signatures: secp256k1 over `proposal_id`
- Authority node signatures: secp256k1 over `node_id`
- Attestation signatures: secp256k1 over `attestation_id`

Ed25519 is not used anywhere in this system.

---

2. StateTransitionV1

```
pub struct StateTransitionV1 {
    pub schema_version: u16,
    pub domain_separator: [u8; 32],
    pub transition_id: [u8; 32],
    pub pre_state_hash: [u8; 32],
    pub post_state_hash: [u8; 32],
    pub mutation_hash: [u8; 32],
    pub authority_context_hash: [u8; 32],
    pub verdict_hash: [u8; 32],
    pub mutation_engine_hash: [u8; 32],
    pub semantic_version: u32,
    pub codec_version: u32,
    pub governor_id: [u8; 32],
    pub governor_signature: Vec<u8>,
    pub epoch: u64,
    pub sequence: u64,
    pub created_at: u64,
}
```

Canonical transition hash:

```
transition_id =
  keccak256(
    abi.encode(
      domain_separator,
      schema_version,
      pre_state_hash,
      post_state_hash,
      mutation_hash,
      authority_context_hash,
      verdict_hash,
      mutation_engine_hash,
      semantic_version,
      codec_version,
      governor_id,
      epoch,
      sequence,
      created_at
    )
  )
```

`governor_signature` is not included in `transition_id`. The Governor signs `transition_id` using secp256k1.

---

3. MutationProposalV1

```
pub struct MutationProposalV1 {
    pub schema_version: u16,
    pub proposal_id: [u8; 32],
    pub proposer_id: [u8; 32],
    pub target_agent_id: [u8; 32],
    pub mutation_type: u32,
    pub payload_hash: [u8; 32],
    pub semantic_version_required: u32,
    pub codec_version_required: u32,
    pub nonce: u64,
    pub created_at: u64,
    pub expires_at: u64,
    pub proposer_signature: Vec<u8>,
}
```

`payload` is not a field of `MutationProposalV1`. Raw payload bytes are stored in content-addressed storage. The Court receives them separately and asserts:

```
keccak256(payload_bytes) == proposal.payload_hash
```

Canonical proposal hash:

```
proposal_id =
  keccak256(
    abi.encode(
      schema_version,
      proposer_id,
      target_agent_id,
      mutation_type,
      payload_hash,
      semantic_version_required,
      codec_version_required,
      nonce,
      created_at,
      expires_at
    )
```

)
```

`proposer_signature` signs `proposal_id` using secp256k1. Not included in `proposal_id`.

### 3.1 Mutation Types

| Code | Name | Description |
|---|---|---|
| `0x0001` | `GENOME_PATCH` | Structured patch to one or more genome fields |
| `0x0002` | `GENOME_REPLACE` | Full genome replacement (requires elevated authority) |
| `0x0003` | `CAPABILITY_GRANT` | Adds a capability flag to the agent |
| `0x0004` | `CAPABILITY_REVOKE` | Removes a capability flag |
| `0x0005` | `ATTESTATION_ATTACH` | Attaches a signed attestation to the agent record |
| `0x0006` | `LINEAGE_FORK` | Forks agent into a new lineage (immutable after) |
| `0x00FF` | `TOMBSTONE` | Terminates agent; no further mutations allowed |

Unknown mutation type codes → `CourtOutcome::InvalidMutationType`.

This is distinct from version errors. Do not conflate them.

### 3.2 Version Checks

- Unsupported `semantic_version_required` → `CourtOutcome::InvalidSemanticVersion`
- Unsupported `codec_version_required` → `CourtOutcome::InvalidCodecVersion`

### 3.3 Expiry

A proposal is expired if `expires_at < transition.created_at`.

On live execution: expired proposals → `CourtOutcome::InvalidAuthority`.

On historical replay: the Court does not re-reject expired proposals. It records them via `AuthorityReview.expired_attestation_flags` and continues verification.

---

## 4. AuthorityContextV1
```

```
pub struct AuthorityContextV1 {
    pub schema_version: u16,
    pub context_id: [u8; 32],
    pub governor_id: [u8; 32],
    pub authority_chain: Vec<AuthorityNode>,
    pub policy_hash: [u8; 32],
    pub policy: Vec<u8>,
    pub attestations: Vec<AttestationV1>,
    pub epoch: u64,
    pub created_at: u64,
}

pub struct AuthorityNode {
    pub node_id: [u8; 32],
    pub role: u32,
    pub delegation_depth: u8,
    pub node_signature: Vec<u8>,
}

pub struct AttestationV1 {
    pub attestation_id: [u8; 32],
    pub attester_id: [u8; 32],
    pub subject_id: [u8; 32],
    pub claim_hash: [u8; 32],
    pub valid_from: u64,
    pub valid_until: u64,
    pub attester_signature: Vec<u8>,
}
```

Canonical context hash:

```
context_id =
  keccak256(
    abi.encode(
      schema_version,
      governor_id,
      hash_authority_chain(authority_chain),
      policy_hash,
      hash_attestations(attestations),
      epoch,
      created_at
    )
  )
```

Where:

```
hash_authority_chain(chain) =
  keccak256(abi.encode(
    for each node in order: (node_id, role, delegation_depth)
    // node_signature excluded
  ))

hash_attestations(attestations) =
  keccak256(abi.encode(
    for each attestation in order:
      (attestation_id, attester_id, subject_id, claim_hash, valid_from, valid_unt
    // attester_signature excluded
  ))
```

Signatures are excluded from all content hashes.

- Each `AuthorityNode` signs its own `node_id` using secp256k1.
- Each `AttestationV1` signer signs its own `attestation_id` using secp256k1.

```
policy_hash = keccak256(policy)
```

The Replay Court does not interpret `policy`. It passes `policy` bytes to the mutation engine WASM as part of the context argument. Policy validity is entirely the mutation engine's responsibility.

4.1 Authority Roles

Code	Role	Max Delegation Depth
`0x01`	`ROOT_GOVERNOR`	0 (self-signing only)
`0x02`	`DOMAIN_GOVERNOR`	1
`0x03`	`AGENT_SUPERVISOR`	2
`0x04`	`PROPOSER`	3

`delegation_depth` must decrease monotonically down the chain.

Any non-monotonic depth in the chain → `CourtOutcome::InvalidAuthority`.

---

5. VerdictV1 (Fully Defined)

```
pub struct VerdictV1 {
    pub schema_version: u16,
    pub outcome: u32,
    pub reason_code: u32,
    pub flags: u32,
    pub authority_context_hash: [u8; 32],
    pub mutation_hash: [u8; 32],
    pub created_at: u64,
}
```

Canonical verdict hash:

```
verdict_hash =
  keccak256(
    abi.encode(
      schema_version,
      outcome,
      reason_code,
      flags,
      authority_context_hash,
      mutation_hash,
      created_at
    )
  )
```

Verdict outcome codes:

Code	Name
0	Allow
1	Deny
2	DenyExpired
3	DenyInsufficientAuthority
4	DenyPolicyViolation

`verdict_hash` is included in `StateTransitionV1` and verified during replay.

---

6. CourtOutcome (Complete Enum)

```
pub enum CourtOutcome {
    TransitionVerified    = 0,
    TransitionChallenged  = 1,
    InvalidSignature      = 2,
    InvalidPostState      = 3,
    InvalidAuthority      = 4,
    InvalidMutationEngine = 5,
    InvalidSemanticVersion = 6,
    InvalidCodecVersion   = 7,
    InvalidMutationType   = 8,
    InvalidPreState       = 9,
    InvalidProposalHash   = 10,
    InvalidContextHash    = 11,
    ReplayUnavailable     = 12,
}
```

Error mapping rules:

- Unknown `mutation_type` code → `InvalidMutationType` (8)
- Unsupported `semantic_version_required` → `InvalidSemanticVersion` (6)
- Unsupported `codec_version_required` → `InvalidCodecVersion` (7)
- These three cases must never be conflated.

---

7. AuthorityReview

```
pub struct AuthorityReview {
    pub valid_at_execution: bool,
    pub valid_under_current_law: bool,
    pub decayed_reputation_score: i32,
    pub expired_attestation_flags: u32,
    pub review_required: bool,
}
```

This enables authority decay without rewriting history:

- `valid_at_execution` : was this authority chain valid at the time of the transition?
- `valid_under_current_law` : would this authority chain be valid today?

The system can evolve policy without falsifying the past.

---

8. Mutation Engine WASM Interface

The Replay Court loads `mutation_engine.wasm` by exact content hash:

```
mutation_engine_hash = keccak256(wasm_bytes)
```

Any byte-level mismatch → `CourtOutcome::InvalidMutationEngine`.

8.1 Required Export

```
apply_mutation(
    pre_state_ptr: u32,
    pre_state_len: u32,
    proposal_ptr: u32,
    proposal_len: u32,
    context_ptr: u32,
    context_len: u32,
    out_ptr: u32
) -> u32
```

Output must be canonical ABI bytes for `AgentGenomeStateV2`.

8.2 Forbidden Capabilities

The WASM module must not use:

- Filesystem access
- Network access
- Clock or timestamp functions
- Randomness
- Environment variables
- Floating point operations
- Threading
- Any host callback except `keccak256`

8.3 Allowed Import

```
keccak256(input_ptr: u32, input_len: u32, out_ptr: u32) -> u32
```

This is the only permitted host import. Execution is pure.

---

9. Hash Function Definitions

All canonical hash functions for this system:

```
// State
state_hash(state: AgentGenomeStateV2) -> [u8; 32]
  = keccak256(canonical_abi_encode(state))

// Proposal (by reference to pre-computed proposal_id)
hash_mutation(proposal: MutationProposalV1) -> [u8; 32]
  = proposal.proposal_id

// Context (by reference to pre-computed context_id)
hash_authority_context(ctx: AuthorityContextV1) -> [u8; 32]
  = ctx.context_id

// Verdict (full field set — v0.3 correction over v0.2)
hash_verdict(verdict: VerdictV1) -> [u8; 32]
  = keccak256(abi.encode(
      verdict.schema_version,
      verdict.outcome,
      verdict.reason_code,
      verdict.flags,
      verdict.authority_context_hash,
      verdict.mutation_hash,
      verdict.created_at
    ))

// Payload (loaded separately from content-addressed storage)
hash_payload(payload_bytes: &[u8]) -> [u8; 32]
  = keccak256(payload_bytes)
```

`canonical_abi_encode` is strict Ethereum ABI encoding:

- No padding variation
- Field order is struct declaration order
- No optional field omission
- Deterministic across all implementations

---

10. Core Execution Invariants (Frozen)

These are the laws of the system. Every invariant must hold for `TransitionVerified`.

```
1. proposal_id    == keccak256(abi.encode(proposal_fields_without_signature))
2. context_id     == keccak256(abi.encode(context_fields_without_signatures))
3. transition_id  == keccak256(abi.encode(transition_fields_without_signature))
4. payload_hash   == keccak256(payload_bytes)
5. mutation_engine_hash == keccak256(wasm_bytes)
6. post_state_hash == state_hash(wasm_apply_mutation(pre_state, proposal, context
7. governor_signature verifies against transition_id using secp256k1
```

If any invariant fails, the transition is not verified. There is no partial verification.

---

11. Verification Algorithm

```
fn verify_transition(
    transition: StateTransitionV1,
    pre_state: AgentGenomeStateV2,
    proposal: MutationProposalV1,
    payload_bytes: &[u8],
    authority_context: AuthorityContextV1,
    mutation_engine_wasm: &[u8],
) -> CourtOutcome {

    // Invariant 5: engine hash
    if keccak256(mutation_engine_wasm) != transition.mutation_engine_hash {
        return CourtOutcome::InvalidMutationEngine;
    }

    // Invariant 4: payload integrity
    if keccak256(payload_bytes) != proposal.payload_hash {
        return CourtOutcome::InvalidProposalHash;
    }

    // Invariant 1: proposal self-consistency
    if hash_proposal_fields(proposal) != proposal.proposal_id {
        return CourtOutcome::InvalidProposalHash;
    }

    // Invariant 2: context self-consistency
    if hash_context_fields(authority_context) != authority_context.context_id {
        return CourtOutcome::InvalidContextHash;
```

```
}

// Version checks (before any state execution)
if !supported_semantic_version(proposal.semantic_version_required) {
    return CourtOutcome::InvalidSemanticVersion;
}
if !supported_codec_version(proposal.codec_version_required) {
    return CourtOutcome::InvalidCodecVersion;
}
if !known_mutation_type(proposal.mutation_type) {
    return CourtOutcome::InvalidMutationType;
}

// Pre-state hash
let computed_pre_hash = state_hash(pre_state);
if computed_pre_hash != transition.pre_state_hash {
    return CourtOutcome::InvalidPreState;
}

// Mutation hash
if proposal.proposal_id != transition.mutation_hash {
    return CourtOutcome::InvalidProposalHash;
}

// Context hash
if authority_context.context_id != transition.authority_context_hash {
    return CourtOutcome::InvalidContextHash;
}

// Authority verdict
let verdict = check_authority(pre_state, proposal, authority_context);
let computed_verdict_hash = hash_verdict(verdict);
if computed_verdict_hash != transition.verdict_hash {
    return CourtOutcome::InvalidAuthority;
}
if verdict.outcome != VerdictOutcome::Allow {
    return CourtOutcome::TransitionChallenged;
}

// WASM execution
let post_state_bytes = wasm_apply_mutation(
    mutation_engine_wasm,
    pre_state,
    proposal,
    payload_bytes,
    authority_context,
);
```

```
    let computed_post_hash = state_hash_from_bytes(post_state_bytes);
    if computed_post_hash != transition.post_state_hash {
        return CourtOutcome::InvalidPostState;
    }

    // Invariant 3: transition self-consistency
    let computed_transition_id = hash_transition_fields(transition);
    if computed_transition_id != transition.transition_id {
        return CourtOutcome::InvalidSignature; // tampered before signing
    }

    // Invariant 7: governor signature
    if !verify_secp256k1(
        transition.governor_id,
        transition.transition_id,
        transition.governor_signature,
    ) {
        return CourtOutcome::InvalidSignature;
    }

    CourtOutcome::TransitionVerified
}
```

---

12. Challenge Protocol

```
pub enum CourtOutcome { ... } // see §6
```

Challenge flow:

```
observer submits challenge
  → bond locked
  → Replay Court re-executes transition with stored inputs
  → if any CourtOutcome != TransitionVerified:
      challenger wins bond, transition flagged
  → if CourtOutcome == TransitionVerified:
      challenger loses bond, transition confirmed
```

No human judgment at this layer. Only deterministic re-execution.

The Court does not take positions. It produces outcomes from inputs.

---

13. Authority Decay

Authority may decay over time without rewriting history.

The Court produces `AuthorityReview` alongside `CourtOutcome`:

```
pub struct AuthorityReview {
    pub valid_at_execution: bool,
    pub valid_under_current_law: bool,
    pub decayed_reputation_score: i32,
    pub expired_attestation_flags: u32,
    pub review_required: bool,
}
```

A transition may be `TransitionVerified` (was valid when executed) while simultaneously having `valid_under_current_law = false` (authority has since expired or been revoked).

These are two distinct facts. The Court records both.

---

14. Normative Test Vector

This is the canonical conformance test. Any implementation must produce identical results.

14.1 Happy Path

Inputs:

```
domain_separator: 0x0000...0001  (32 bytes)
epoch: 1
transition.sequence: 1
transition.created_at: 1700000100

pre_state (AgentGenomeStateV2):
  agent_id:          0xaaaa...0001
  version:           1
  capability_flags:  0x00000000
  genome_fields:     {} (empty)
  lineage_root:      0x0000...0000

proposal (MutationProposalV1):
  schema_version:           1
  proposer_id:              0xbbbb...0002
  target_agent_id:          0xaaaa...0001
  mutation_type:            0x0003  (CAPABILITY_GRANT)
```

```
  payload_hash:              keccak256(abi.encode(0x0001))
  semantic_version_required: 1
  codec_version_required:    1
  nonce:                     1
  created_at:                1700000000
  expires_at:                1800000000

payload_bytes: abi.encode(flag=0x0001)

authority_context (AuthorityContextV1):
  schema_version: 1
  governor_id:    0xcccc...0003
  authority_chain:
    [0] node_id: 0xcccc...0003, role: 0x01 (ROOT_GOVERNOR), delegation_depth: 0
    [1] node_id: 0xbbbb...0002, role: 0x04 (PROPOSER),      delegation_depth: 1
  policy_hash:  keccak256(policy_bytes)
  policy:       <abi-encoded: allow CAPABILITY_GRANT of flag=0x0001 to any agent>
  attestations: []
  epoch:        1
  created_at:   1700000000
```

Expected outputs:

```
payload_hash verified:       true
proposal_id verified:        true
context_id verified:         true
authority verdict:           Allow (outcome=0, reason_code=0, flags=0)
post_state.capability_flags: 0x00000001
court_outcome:               TransitionVerified (0)

authority_review:
  valid_at_execution:        true
  valid_under_current_law:   true
  decayed_reputation_score:  0
  expired_attestation_flags: 0
  review_required:           false
```

14.2 Failure Vectors (All Must Pass)

#	Perturbation	Expected CourtOutcome
F1	Flip one byte of `mutation_engine_wasm`	`InvalidMutationEngine` (5)
F2	`expires_at = created_at - 1`	`InvalidAuthority` (4)

F3	`delegation_depth[1] = 0` (non-monotonic)	`InvalidAuthority` (4)
F4	`mutation_type = 0x00FF` (TOMBSTONE, policy disallows)	`TransitionChallenged` (1)
F5	Corrupt `governor_signature`	`InvalidSignature` (2)
F6	`post_state_hash` set to `0x0000...0000`	`InvalidPostState` (3)
F7	`pre_state_hash` set to wrong value	`InvalidPreState` (9)
F8	`mutation_type = 0xDEAD` (unknown type)	`InvalidMutationType` (8)
F9	`semantic_version_required = 999` (unsupported)	`InvalidSemanticVersion` (6)
F10	`codec_version_required = 999` (unsupported)	`InvalidCodecVersion` (7)
F11	`payload_bytes` tampered (hash mismatch)	`InvalidProposalHash` (10)
F12	`context_id` field set to `0x0000...0000`	`InvalidContextHash` (11)

Each failure case must:

1. Produce exactly the listed `CourtOutcome` code
2. Produce no state change
3. Be reproducible across all conforming implementations

---

15. Implementation Order

Build in this sequence. Do not skip ahead.

Step	Deliverable	Gate
1	`AgentGenomeStateV2` canonical ABI schema	—
2	All hash functions (§9)	Step 1 complete
3	secp256k1 signature generation and verification	Step 2 complete
4	Happy-path test vector (§14.1) with concrete byte values	Step 3 complete

5	All 12 failure vectors (§14.2) pass	Step 4 complete
6	Reference WASM mutation engine	Step 5 complete
7	Challenge protocol	Step 6 complete

Do not begin WASM development until Steps 1–5 pass. Hash and signature correctness is the foundation. WASM correctness is built on top.

---

16. Open Items (Tracking)

Item	Status
`AgentGenomeStateV2` full ABI schema	Not yet defined
Concrete byte values for §14.1 hashes	Pending Step 4
Reference `mutation_engine_v1_test.wasm`	Pending Step 6
`VerdictV1` outcome codes extended (reason_code registry)	Not yet defined
Reputation decay scoring formula	Not yet defined
Bond sizing for challenge protocol	Not yet defined

---

v0.3 — all v0.1 and v0.2 content superseded. No open ambiguities remain at the protocol layer.